A Review of Cryptanalysis Techniques for Lattice Cryptography using Machine Learning

Agha Aghayev[1], Yadigar Imamverdiyev[1]
Oruj Orujlu[1]

•

[1]Azerbaijan Technical University, Baku, Azerbaijan

Abstract

Machine Learning has shown an increasing trend due to its wide range of applications. As developments made in Machine Learning, numerous applications have been successfully applied to diverse fields, including optimization, mathematics, and security. On the other hand, Cryptography has become the bedrock for all of daily applications, user privacy, and government. The security of asymmetric cryptographic algorithms rely on hardness assumptions and the recent cryptanalysis methods aim to advance our understanding on such problems using Machine Learning methods. Recent improvements in quantum cryptanalysis have shown that contemporary approaches whose security rely on factorization, discrete-log problem, etc. all the number theoretical assumptions will be insecure against large-scale quantum computers. Researchers have investigated hardness assumptions for quantum-resistance, and their cryptanalysis with machine learning is a new area of interest. Hence, we provide a review of machine learning algorithms for post-quantum cryptography, with a focus on lattice-based cryptography. We provide the basic quantum-proof hardness assumptions defined on lattices, review the machine learning frameworks used in their cryptanalysis. Our work represents one of the first comprehensive reviews of machine learning–based cryptanalysis techniques, with a particular focus on lattice-based cryptography.

Keywords: Cryptography, Machine learning, Cryptanalysis, Learning With Errors (LWE), Shortest Vector Problem(SVP), Lattice Reduction, Post-quantum cryptography

I. Introduction

In recent years, *Machine Learning (ML)* has been evolved as one of the most transformative trend of the digital era. The concept behind many machine learning algorithms lies in extracting meaningful patterns and insights from some amount of data collected for a specific problem, to address automation process for complex decision-making processes. Its dominance has spread over many scientific and industrial domain, examples include healthcare diagnostics for better predictions, financial forecasting for improving income, autonomous systems, cybersecurity to prevent vulnerabilities, and natural language understanding. The performance of machine learning models rely on data and computational power, supported with algorithmic advancements.

Traditional ML techniques, have shown significant improvement for our understanding of machine learning models and they benefit from cost effective computation, and have laid the foundation for modern intelligent systems. These algorithms are often effective for structured data and moderate-scale tasks, and often require some knowledge on domain of the problem. Many tasks

often include deep hierarchical representations required for high-dimensional problems. This limitation has been partially resolved with introduction of *neural networks*, which abstractly mimic learning process of the human brain. This trend has also affected the development of computational power, enabling training of models with millions (and later billions) of parameters for image recognition, speech processing and understanding, etc.

The *Deep Neural Networks (DNNs)* [1] even improved our performance by including many hidden layers, and started a new era of more sophisticated architectures, including convolutional networks for spatial data, recurrent networks for time-based data, and the state-of-the art transformer-based [2] models for language learning. The result of these developments led to the *Large Language Models (LLMs)* such as GPT [3], and they have shown improved capabilities in reasoning, translation, and generative tasks. The attention mechanism has marked a significant step toward artificial general intelligence and improve them to be multi-purpose models. Beyond the first wave of applications on natural language processing, many LLM-based architectures are now used for multimodal data, including vision, code, and scientific modeling.

Parallel to the development of machine learning techniques, another young domain—*cryptography*— has gained significant attention in recent years and served as the bedrock of information security. In the digital era, billions of transactions happen every day, confidentiality and integrity of data carry major importance for secure systems, for user privacy and preventing malicious attackers. Many applications include hybrid encryption schemes, where the secret key is first shared with some public key architecture, which is often time consuming, and the shared secret key is used for fast encryption via symmetric primitives. All of the asymmetric key primitives rely on some sort of *mathematical hard problem*, which is hard to solve but easy to verify when given a solution. Classical cryptographic schemes, such as RSA encryption scheme [4], Diffie-Hellman key exchange [5] rely on the computational hardness of integer factorization and discrete-logarithm problems. However, in 1992, Peter Shor has introduced an polynomial-time *quantum* algorithm [6] which can efficiently break both factorization and discrete-log problems, making today's standards vulnerable to future large-scale quantum computers. This posses the development of *Post-Quantum Cryptography (PQC)*—the cryptographic algorithms designed to resist attacks from quantum computers.

Among various PQC candidates, *lattice-based cryptography* has gained attention due to its fast arithmetic and parallelizable property. Some of the of well-studied lattice problems, such as the Shortest Vector Problem (SVP), is known to be *NP-hard* [7]. Other known hardness assumptions, such as Learning With Errors (LWE) [8] which is widely used in constructions, is proven that solving this hard problem in average-case is as hard as other related lattice problems in **worst-case**. Until today, there is not known any classical or quantum algorithm to solve such problems efficiently. Furthermore, lattice-based constructions also opened doors to richer functionalities, including but not limited to, homomorphic encryption, digital signatures, and identity-based encryption, etc. making them an important candidate for the future of cryptography.

Another important application of lattices is cryptanalysis, so-called lattice reduction algorithms. The main criteria of the reduction algorithm is to improve the orthogonality of the lattice vectors and can be applied to recover a "short" enough vector to break many cryptosystems [9].

The cryptanalysis of lattice-based hardness assumptions via machine learning have become more applicable as we combine huge computation power and effective learning algorithms to learn the secret information within randomness. Despite the fact that it's a new area, ML-based cryptanalysis have been successfully applied to lattice-based hardness assumptions and lattice-reductions techniques. Hence, our paper focuses on reviewing current applications of machine learning for cryptanalytical purposes in lattice-based cryptography. We first review the lattice hardness assumptions, and then provide existing methodologies on use cases of machine learning.

II. Literature review

Since cryptanalysis of lattice-based cryptography using machine learning algorithms from a mathematical perspective is a new area, our work is among the first to review such approaches.

III. Preliminaries and Notations

We denote integers with $\mathbb{Z}$ vectors with bold letter $\mathbf{v}$ and matrices with uppercase letters $\mathbf{B}$. A lattice $\mathcal{L} \subset \mathbb{Z}^n$ is the set of all integer combinations of linearly independent vectors $\mathbf{v}_1, \ldots, \mathbf{v}_m$:

$$\mathcal{L} = \left\{ \sum_{i=1}^{m} a_i \mathbf{v}_i \mid a_i \in \mathbb{Z} \right\} = \mathcal{L}(\mathbf{B}),$$

where $\mathbf{B} = [\mathbf{v}_1 \ \cdots \ \mathbf{v}_m]$ is the basis of lattice. We define a continuous n-dimensional Gaussian function centered at $\mathbf{c}$ the origin with parameter $s > 0$ and for all $\mathbf{x} \in \mathbb{R}^n$, define

$$\rho_{s,\mathbf{c}}(\mathbf{x}) = \exp(-\pi \frac{\| \mathbf{x} - \mathbf{c} \|^2}{s^2}).$$

For any $c \in \mathbb{R}^n$, real $s > 0$, and n-dimensional lattice Λ, define the discrete Gaussian distribution over Λ as:

$$\forall \mathbf{x} \in \Lambda, D_{\Lambda,s,\mathbf{c}}(\mathbf{x}) = \frac{\rho_{s,\mathbf{c}}(\mathbf{x})}{\rho_{s,\mathbf{c}}(\Lambda)}.$$

Now we will define the key lattice-based hardness assumptions.

Assumption 1 - Shortest Vector Problem(SVP) [7]**:** Given a lattice $\Lambda \subset \mathbb{Z}^n$ defined by a basis $B = \{b_1, b_2, \ldots, b_n\}$, find a non-zero lattice vector $v \in \Lambda$ such that

$$\|v\| = \min \{ \|u\| : u \in \Lambda \setminus \{0\} \}.$$

In other words, SVP asks for the shortest non-zero vector in the lattice with respect to the Euclidean norm. There are many norms can be defined, we will stick to Euclidian norm.

Assumption 2 – Learning With Errors Problem (LWE) [8]: Let n be a positive integer, q be a modulus, and χ be an error distribution (often Gaussian) over the integers. In matrix notation, given polynomially many samples (A, b), where A is a random matrix of size m × n with entries in $\mathbb{Z}_q$, and $\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$ (mod q), for some secret vector $s \in \mathbb{Z}_q^n$ chosen uniformly at random, and an error vector $e \in \mathbb{Z}_q^m$ whose entries are drawn from the distribution χ. Given ($\mathbf{A}$, $\mathbf{b}$), find $\mathbf{s}$.

Quality of lattice bases. Randomly defined lattice bases tend to have poor qualities for finding a short vector or the closes vector to a given point, and often used as public key for cryptosystems. The knowledge of a "good" basis is often used as a secret key. The orthogonality of basis vectors defines its quality, the more orthogonal, the more useful for finding short vectors in a lattice. Given a basis, we can apply lattice reduction techniques such as LLL [10] and BKZ [11] algorithms. We will also review machine learning techniques to improve or predict lattice reduction techniques.

IV. Review of Approaches

In this section, we will explain the machine learning techniques for attacking problems.

a. SALSA attacks

The Learning With Errors (LWE) problem has been widely used cryptographic applications, including but not limited to, homomorphic encryption, secure messaging, functional encryption,

identity-based encryption, etc. In nutshell, the problem asks for finding a secret solution given a set of equations which are perturbed with some errors, which can be uniform, Gaussian, binary, etc. The existing cryptanalysis approaches against this problem include three main categories: combinatorial attacks, algebraic attacks (Arora-Ge style) and lattice reduction techniques, such as LLL and BKZ algorithm. All of attack algorithms either have exponential computational complexity or exponential memory requirements. More recently, the first machine learning algorithm was applied to LWE problem by Wenger et al [12], where they introduce **SALSA**, a method for **S**ecret-recovery **A**ttacks on **L**WE via **S**eq2Seq models with **A**ttention. The hardness of the LWE problem lies in adding random noise to hide secret vector, and the machine learning approach aims to learn secret vector given enough number of samples. Moreover, this first approach is only applicable to find *sparse binary (or ternary)* secrets, where the coefficients of **s** are sampled from {0,1} (or {-1, 0, 1} for ternary). Such family of secret key parameters are used in practical applications like libraries and homomorphic encryption [13]. The modular arithmetic is the main obstacle for machine learning based attacks, since without modula operation, it would be a simple linear regression task which could be solved with high accuracy. The transformer model is fed with plenty of LWE samples, and trained to predict b values from **a** by minimizing cross-entropy loss between model prediction b' and b. The authors employ a two-layer encoder (1024 dimensions, 32 attention heads, with the second layer iterated twice) and a two-layer decoder (512 dimensions, 8 heads, with the second layer iterated eight times). The model is trained using the Adam optimizer with a learning rate of $10^{-5}$and 8,000 warm-up steps. After each epoch, they evaluate the accuracy on the test LWE samples within some *tolerance* τ, for instance, $|b\text{-}\tilde{b}| \leq \tau q$. **Direct Secret Recovery** and **Distinguisher Secret Recovery** are the two core techniques proposed in SALSA for extracting the secret vector **s**:

- **Direct Secret Recovery** treats the problem analogously to a chosen-plaintext attack. For each coordinate i, the model M is queried with a specially crafted input $\mathbf{a}_i = K\mathbf{e}_i$ (where only the i-th component is nonzero). The model's output $\tilde{b} = M(\mathbf{a}_i)$ is analyzed: small predicted values indicate $\mathbf{s}_i = 0$, and large values indicate $\mathbf{s}_i = 1$. Repeating this for all coordinates and binarizing the predictions yields a full secret guess.
- **Distinguisher Secret Recovery** builds on the *decision-LWE* formulation. It compares the model's responses to real LWE samples and to uniformly random samples. By modifying each feature $a_i$with random offsets and measuring how the model's outputs differ between the two cases, the algorithm infers whether each bit of **s** contributes to distinguishing power—recovering **s** bit by bit when the model's accuracy exceeds a threshold (≈30%).

Secret Verification. After computing some secret vectors $\tilde{\mathbf{s}}$, the residuals $r = \mathbf{a} \cdot \tilde{\mathbf{s}} - b \bmod q$ are computed over LWE samples. If $\tilde{\mathbf{s}} = \mathbf{s}$, the residuals will have a small standard deviation σ; otherwise, they appear nearly uniform over $\mathbb{Z}_q$with deviation around $q/\sqrt{12}$.

Data Generation. The LWE samples are generated with fixed modulus q=251 and a base element for representing numbers B=81, where the errors sampled from zero-mean discrete Gaussian with σ =3. The hamming weight h is computed with a secret density variable d, where the $h = nd$. The experiments are evaluated with the values of n in $[30:128]$ with density $0.002 \leq d \leq 0.15$. They also show the attack on ring version of the LWE problem, RLWE with different parameter settings where n is a power of two. The model training is moderately small, for example, for n=128 the model took 46 hours and can recover up to binary secrets with 3 Hamming weight (d=0.02). An important observation is that the model is able to **learn** the secret as soon as the training loss drops. The model requires around 2^{20} samples, which is never required in practice. However, the samples do not need to be linearly independent, and they can generate new training samples by only using 100 original samples.

They have recently improved the attack success by simplifying the model with lighter models, added data preparations using lattice reductions to improve data improving the orthogonality of samples, enabling success on real world size LWE samples with improved Hamming weight.

SALSA PICANTE [14]. The improved version starts with m=4n original samples, and subsamples n out of m samples, resulting in matrices A in $Z\^\{n\ x\ n\}$ and applies preprocessing using lattice

reduction (BKZ) on following matrix

$$\begin{bmatrix} w\mathrm{I}_n & \mathbf{A}_{n\times n} \\ 0 & q\mathrm{I}_n \end{bmatrix}$$

Since the lattice reduction is a linear transformation, we can also represent it as following

$$[\mathbf{R}_{2n\times 2n} \quad \mathbf{C}_{2n\times 2n}] \begin{bmatrix} w\mathrm{I}_n & \mathbf{A}_{n\times n} \\ 0 & q\mathrm{I}_n \end{bmatrix} = [w\mathbf{R} \quad \mathbf{RA} + q\mathbf{C}]$$

where **R** is the transformation applied to matrix **A**, hence it is also applied to b and the error term e. If the error term grows too much, the problem will become unsolvable and there should be a trade-off between reduction quality and error growth, controlled by parameter w. They represent values in a bigger base $B \geq \sqrt{q}$, train the model as a supervised multi-classification task. The model is highly inspired by original SALSA [12], where they also employ a similar strategy for secret recovery and verification. The LWE samples are generated with standard deviation σ =3,
and $h \approx n/10$ for hamming weight.

Results. For $n \leq 300$, Picante consistently outperforms SALSA on recoving LWE secrets.
At $n = 350$, Picante recovers secrets with $h = 60$(≈17% sparsity), attributed to improved preprocessing and works for even smaller moduli q (the smaller moduli, the harder problem becomes). Surprisingly,the total cost is dominated by the data preprocessing stage, which is CPU-intensive but fully parallelizable. The preprocessing cost per matrix grows from 0.01 CPU hours at $n = 80$ to 194 CPU hours at $n = 350$. Model training time scales roughly linearly with dimension, ranging from 42 minutes per epoch for $n = 80$ to 105 minutes for $n = 350$ on a single NVIDIA V100 GPU. Secret recovery requires less than 10% of the total runtime.

Building upon [12] and [14], SALSA VERDE [15] improves to breaking LWE instances with sparse ternary and even narrow Gaussian secrets, larger dimension ($n = 512$) and smaller moduli q. The authors introduce new recovery techniques, which lead to reduced preprocessing and a new notion – **NoMod** – a framework for analyzing success of ML-based attacks on LWE, also benefits from lighter models with *encoder only* model and attacks medium-to-hard LWE problems. It was observed that Seq2seq models can have longer outputs than inputs, but it is not necessary for LWE problem. They utilize a similar strategy of distinguisher for recovering secrets: measures the summed absolute difference between model predictions at test vectors $a_{\text{test}}$and $a_{\text{test}} + Ke_i$(for random $K \in (0.3q, 0.7q)$) to score each secret coordinate—larger scores indicate likely nonzero positions, and they adapt this idea for ternary and Gaussian secrets. Overall approach is similar to PICANTE [14], though with improved results: for $n = 256$, the attack recovers binary and ternary secrets with 10% sparsity(with modulus $log_2\, q = 20$) and with 3% sparsity (for $log_2 q = 12$). This work finally breaks a real size LWE problem: for $n = 512$, $log_2\, q = 41$, the model recovers binary and ternary secrets with ≥ 11% sparsity. Each epoch requires 1.5/1.6/2.5 hours for n = 256/350/512, respectively for training VERDE models. Additionally, they report outperforming efficiency uSVP attacks via Kannan's embedding [16], with more computational parallelized resources.
Finally, Stevens et al. [17] introduced a much powerful attack on LWE problem, with more improved preprocessing techniques, an angular embedding technique for better representation of the problem and *first* use of pre-training for ML-based attacks. This attack also requires $4n$ samples, applies reductions on subsampled matrices of $\mathbf{A}_{n\times n}$

$$\begin{bmatrix} 0 & q\mathrm{I}_n \\ w\mathrm{I}_n & \mathbf{A}_{n\times n} \end{bmatrix}$$

with a careful selection of reduction parameter w to keep the balance of error increase. In previous approaches, they have represented all numbers in base B , which resulted in two large vocabularies in LLM models. Hence, they introduce an **angular embedding** to better capture the modular structure of the problem while encoding each integer with a single token. An integer $a \in \mathbb{Z}_q$ is

mapped to an angle $\frac{2\pi a}{q}$ and represented as the point $\left(\sin\left(\frac{2\pi a}{q}\right), \cos\left(\frac{2\pi a}{q}\right)\right)$on the unit circle. These representations are then linearly projected into $\mathbb{R}^d$, and model outputs are decoded back into $\mathbb{R}^2$using another projection. Training is guided by an L2 loss between the predicted point and the true target point on the unit circle, and experiments show that improved recovery accuracy and significantly higher computational efficiency compared to traditional vocabulary-based embeddings. The new approach with angular embeddings can successfully attack $n = 768$ and $n = 1024$ dimensional secrets (where $n = 512$ was the maximum dimension with previous approaches). The model can recover up to $h = 9$ for both $n = 768$ and $n = 1024$ settings in less than 24 hours of training on a single 32GB V100 GPU, and they show that the encoder-decoder model fails to attack such dimensions in reasonable time. They have also provided experiments with fewer training samples $N = [100K, 300K, 1M, 3M]$, and the model can attack LWE problems $n = 512$, $log_2 q = 41$ with binary secrets and hamming weight h = 30-45, with additional cost complexity. Experiments show that even $N = 1M$ training samples can achieve the same attack recovery in terms of hamming weight with respect to $N = 4M$ training samples. They show that pre-training can reduce the preprocessing greatly reduces the number of samples needed for recovery (from 1.7M to 409K on average), but makes secret recovery slightly slower. The following table summarizes the attack results of above approaches for the LWE instances with the largest dimension.

Method	n	$log_2 q$	Secret	h / Sparsity	Original samples	Generated Samples	Time
SALSA	128	8	$Bin\ /\ Ter$	$h \approx 3$ (~2%)	100	$1\ M$	~46 h
PICANTE	350	32	$Binary$	$h \approx 60$ (~17%)	$4n$	$4\ M$	$42 - 105\ min/ep$
VERDE	512	41	$Bin\ /\ Ter\ /\ Gauss$	~11%	$4n$	$4\ M$	$1.5 - 2.5\ h/ep$
FRESCA	1024	41	$Bin\ /\ Ter$	$h = 9$ (~1%)	$4n$	$1 - 4\ M$	$< 24\ h$

Sexana et al. [18] recently introduced TAPAS, a publicly available dataset of LWE samples designed to simplify experimentation and facilitate attacks on the LWE problem for non-experts. The deterministic version of LWE- the Learning With Rounding problem has common properties with LWE problem, and it is an open direction to apply machine learning attacks in this direction.

b. Attacks on Shortest Vector Problem

In this section, we review existing attacks on Shortest Vector Problem with machine learning algorithms. The hardness of SVP problem lies in the expensive cost of searching short enough vectors in exponentially large space in security (dimension) parameter, hence finding a shortest vector is NP-hard. The existing algorithms for solving SVP problem is exponential in dimension, making it infeasible for finding shortest vectors in reasonable time. The recent work of Singh et al. [19] introduced an attack on this problem using machine learning algorithm, namely the *K*-Nearest Neighbours (KNN). The KNN is non-parametric classical algorithm that extracts *K* closest training examples to a given input based on a distance metric (the Euclidean distance), and then uses the closest data points for prediction. Despite its simplicity, KNN is computationally expensive for large datasets and can only attack to toy examples of SVP instances. The approach generates lattice vectors from synthetic $2D$, $4D$, and $10D$ dimensional bases with "short" or "long" labels. The K-Nearest Neighbour classifier iteratively halves the dataset and discards long-vector clusters to narrow the search. The method achieved about **78%** accuracy on $2D$, **58%** on $4D$,

and **61%** on $10D$ datasets, however, it suffers from the curse of dimensionality and remaining impractical against real cryptographic (high-dimensional) lattices. Additionally, the problem only provides a lattice bases and asks for smallest vectors, and we will not have labels for long or short vectors. It is also questionable the effectiveness of the algorithm, since we can already compute the shortest vector among given examples. The future work includes more scalable algorithms for larger dimensions and use of reinforcement algorithms.

c. Applications of Machine Learning for Lattice reduction

The lattice reduction algorithms are optimization techniques aiming to improve the orthogonality of vectors in a given lattice bases. The "bad" lattice bases consists of long skewed vectors, which have worse decoding properties. On the other hand, the "good" bases have more orthogonal and shorter vectors, which can be used to efficiently decode lattice points, and can be used for lattice trapdoors [19] and many richer constructions. The LLL algorithm [10], introduced by Lenstra, A. K., Lenstra, H. W., & Lovász was one of the first steps toward this problem. It uses Gram-Schmidt orthogonalization method as a subroutine to iteratively, the key difference is that the reduced basis by LLL algorithm also represent the same lattice. As mentioned earlier, it can also be represented as a linear operations of the basis $\mathbf{B}' = \mathbf{BQ}$, where $\mathbf{Q}$ is a unimodular base whose determinant is ± 1. To measure the orthogonality of a lattice basis, we use following definition

Definition 1. The orthogonality defect of $\mathbf{B} \in GL_n(\mathbb{R})$is

$$\delta(\mathbf{B}) = \frac{\prod_i \| \mathbf{b}_i \|}{| \det(\mathbf{B}) |}$$

The orthogonality defect is shown to be closely related to the SVP problem, where the problem becomes easier with more orthogonal and results in the following computational problem:

Problem 1 (Lattice Reduction). Given a lattice bases $\mathbf{B} \in GL_n(\mathbb{R})$, find a transformation matrix $\mathrm{Q} \in GL_n(\mathbb{Z})$ *such that* minimizes $\delta(\mathbf{BQ})$.

The LLL algorithm can approximately solve lattice reduction problem, and returns a basis $\mathbf{B'} = \mathbf{QB}$ with orthogonality defect bounded by

$$\delta(B') \leq 2^{n(n-1)/4}.$$

The complexity of LLL algorithm is polynomial in n and maximum length of input vectors, and requires $O(n^4 \log \beta)$ operations. Marchetti et al. [20] has given a novel approach for predicting a more orthogonal basis given arbitrary lattice basis, and they show that it is indeed possible to train such a model without examining LLL algorithm. Since the ground truth data (a better basis) is not knowing for many cases of random lattice bases, they employ *self-supervised* machine learning methods using symmetry-aware design principles into their neural network, to ensure that the model consider the geometric structure of the lattice reduction problem. They train model

$$\phi: GL_n(\mathbb{R}) \rightarrow GL_n(\mathbb{Z})$$

to approximate lattice reduction, outputs a unimodular transform matrix $\mathbf{Q}$ for given basis $\mathbf{B}$. The model minimizes a specific loss defined on logarithmic orthogonality defect of the reduced basis $\mathbf{B}' = \mathbf{B}\phi(\mathbf{B})$, defined as

$$L(\mathbf{B}, \theta) = \log \delta(\mathbf{B}') = \sum_i \log \| \mathbf{b}_i' \| - \log | \det(\mathbf{B}') | \geq 0,$$

where θ are the trainable parameters of ϕ. The model output must be with integer entries and invertible unimodular, where they use elements of $SL_n(\mathbb{Z})$ using **Gauss moves**—elementary integer operations with determinant one. So, the output consist of multiplication of multiple Gauss moves and the output matrix will still be a unimodular matrix, which will still yield the same lattice when multiplied with the original base. The network outputs real-valued matrices whose entries are interpreted probabilistically via the **Gumbel-Softmax** trick and discretized using **stochastic rounding**, allowing differentiable sampling of integer values. The overall computational complexity of the proposed method scales as $O(n^3k)$, where $k = O(n)$, making it asymptotically comparable to the classical LLL algorithm.

Experiments. The authors evaluate their method on lattices sampled from four distributions: Uniform Exponential (exponential of a random basis), Convex (generated as random convex combinations of a small number of bases), and Ajtai (duals of discrete uniform matrices). Results show that the model is comparable to classical LLL in many cases, it slightly outperforms LLL on Exponential lattices, while performance is lower on Ajtai lattices due to the wider range of values in the dual basis.

Cheong et al. [22] has investigated improvements on the other lattice reduction BKZ [11] using deep reinforcement learning methods. The BKZ algorithm applies reduction techniques in blocks, and often outperforms LLL algorithm in terms of the quality of the reduction. The bigger block size β results in better reduction quality, tough additional with exponential complexity in block size, hence, it is an important question to find a trade-off between reduction quality and block size. The main approach is to use reinforcement learning for dynamically adapted BKZ block size selection strategies based on reduction data, replacing fixed heuristics to achieve more efficient and instance-specific lattice reductions. The problem is formulated as a Markov decision process, with states representing the current lattice basis $B^{(t)}$, the previous block size β_{t-1}, and an indicator of whether the basis was modified, while actions correspond to selecting a block size $\beta_t \in \{2, \dots, n\}$ or terminating the reduction. The extrinsic reward balances multiple objectives: a time penalty $R_\tau(t) = \tau_t - \tau_{t-1}$, shortest vector improvement $R_\ell(t) = \min_i \| b_i^{(t-1)} \| - \min_i \| b_i^{(t)} \|$, and orthogonality defect reduction $R_\Lambda(t) = \delta(B^{(t-1)}) - \delta(B^{(t)})$, combined as $R_{\text{ext}}(t) = w_\tau R_\tau + w_\ell R_\ell + w_\Lambda R_\Lambda$ for final reward. Intrinsic curiosity-based rewards are derived from forward dynamics and reconstruction errors to encourage exploration. They use an index-parameterized scoring function to score each action, including termination, and select actions via a softmax function, sampling during training and choosing the highest score during evaluation. Transformer encoders process Gram-Schmidt lengths with positional embeddings, and action encoders incorporate both β_t and lattice dimension n. Experiments show the agent learns non-trivial block size sequences, though generalization to higher dimensions remains limited due to reward scaling and exponential runtime growth. They have shown that this approach can outperform Gaussian Heuristics on the well known Knapsack problem.

V. Conclusion

We have provided a review of machine learning techniques for attacking lattice-based cryptography. We investigated attacks on the Learning With Errors (LWE) problem, the Shortest Vector Problem (SVP), and improvements in lattice reduction algorithms. Attacks on LWE problems are promising for sparse secrets with large dimensions, but their

effectiveness on uniform secrets with a full range is not feasible. The only existing SVP attacks consider small-dimension toy examples, requiring further investigation. Lattice reduction algorithms also face challenges with higher dimensions. The future work include improving the algorithms for attacking real-world instances. In the early stages of ML-based attacks, we conclude that these attacks and strategies do not yet raise significant concerns for hard lattice problems.

References

[1] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Networks*, vol. 61, pp. 85–117, 2015.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[3] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018.

[4] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[5] W. Diffie and M. E. Hellman, "New directions in cryptography," in *Democratizing Cryptography: The Work of Whitfield Diffie and Martin Hellman*, pp. 365–390, 2022.

[6] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999.

[7] M. Ajtai, "The shortest vector problem in L2 is NP-hard for randomized reductions (extended abstract)," in *Proc. 30th ACM Symp. Theory of Computing (STOC)*, Dallas, TX, 1998, pp. 10–19.

[8] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *Journal of the ACM (JACM)*, vol. 56, no. 6, pp. 1–40, 2009.

[9] M. Albrecht and L. Ducas, "Lattice attacks on NTRU and LWE: A history of refinements," *Cryptology ePrint Archive*, 2021.

[10] A. K. Lenstra, H. W. Lenstra, and L. Lovász, "Factoring polynomials with rational coefficients," *Math. Ann.*, vol. 261, no. 4, pp. 515–534, 1982.

[11] C. P. Schnorr and M. Euchner, "Lattice basis reduction: Improved practical algorithms and solving subset sum problems," *Mathematical Programming*, vol. 66, no. 1, pp. 181–199, 1994.

[12] E. Wenger, M. Chen, F. Charton, and K. E. Lauter, "SALSA: Attacking lattice cryptography with transformers," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 34981–34994, 2022.

[13] M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, et al., "Homomorphic encryption standard," in *Protecting Privacy through Homomorphic Encryption*, Cham: Springer, 2022, pp. 31–62.

[14] C. Y. Li, J. Sotáková, E. Wenger, M. Malhou, E. Garcelon, F. Charton, and K. Lauter, "Salsapicante: A machine learning attack on LWE with binary secrets," in *Proc. 2023 ACM SIGSAC Conf. Computer and Communications Security*, 2023, pp. 2606–2620.

[15] C. Li, E. Wenger, Z. Allen-Zhu, F. Charton, and K. E. Lauter, "SALSA VERDE: A machine learning attack on LWE with sparse small secrets," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 53343–53361, 2023.

[16] R. Kannan, "Minkowski's convex body theorem and integer programming," *Mathematics of Operations Research*, vol. 12, no. 3, pp. 415–440, 1987.

[17] S. Stevens, E. Wenger, C. Li, N. Nolte, E. Saxena, F. Charton, and K. Lauter, "Salsa fresca:

Angular embeddings and pre-training for ML attacks on Learning with Errors," *arXiv preprint arXiv:2402.01082*, 2024.

[18] E. Saxena, A. Alfarano, F. Charton, E. Wenger, and K. Lauter, "TAPAS: Datasets for learning the Learning with Errors problem," *arXiv preprint arXiv:2510.08797*, 2025.

[19] S. P. Singh, B. K. Chaurasia, T. Tripathi, A. Pal, and S. Gupta, "Attack on lattice shortest vector problem using K-Nearest Neighbour," *Iran Journal of Computer Science*, vol. 7, no. 3, pp. 515–531, 2024.

[20] C. Gentry, C. Peikert, and V. Vaikuntanathan, "Trapdoors for hard lattices and new cryptographic constructions," in *Proc. 40th ACM Symp. Theory of Computing*, 2008, pp. 197–206.

[21] G. L. Marchetti, G. Cesa, K. Pratik, and A. Behboodi, "Neural lattice reduction: A self-supervised geometric deep learning approach," *arXiv preprint arXiv:2311.08170*, 2023.

[22] S. F. Cheong, C. W. Lim, and Y. B. Ti, "Investigating deep reinforcement learning for BKZ lattice reduction," 2025.